

Multiplicação Esparsa Matriz-Vetor para LLMs Podados: Um Formato Customizado Inspirado no MACKO com Layout SoA e Precisão Mista FP16/FP64 em GPU CUDA

Disciplina: Computação de Alto Desempenho — INF, Universidade Federal de Goiás
Professor: Ricardo Augusto Pereira Franco

Resumo

Este trabalho propõe e avalia um formato de armazenamento esparsos customizado — inspirado na filosofia do formato MACKO, com contribuições originais de layout e compressão — para a operação de Multiplicação Esparsa Matriz-Vetor (SpMV) aplicada a matrizes características de Large Language Models (LLMs) podados. O formato proposto, denominado MACKO-SoA-FP16, evolui em três etapas iterativas: (v1) estrutura de chunks AoS com coluna absoluta em int32; (v2) delta encoding com int16 e máscara de validade por bitmask; e (v3) refatoração do layout Array of Structs (AoS) para Structure of Arrays (SoA), com uma variante adicional de precisão mista FP16/FP64. O benchmark foi executado numa NVIDIA RTX 4090 com matrizes quadradas de dimensão $n \in \{4096, 16384, 32768\}$ e esparsidade $\in \{0,30; 0,50; 0,70; 0,85; 0,90\}$, totalizando 15 configurações com 100 amostras cada. O kernel GPU CUDA com warp-per-row no formato SoA atingiu até 1894,59 GB/s de largura de banda efetiva e um speedup de 5× sobre o kernel CSR-CUDA ingênuo na mesma plataforma. A variante FP16 entregou speedups adicionais de até 1,9× sobre a versão FP64 em configurações de baixa esparsidade ($n=4096, s=0,30$), com correteza numérica verificada em todas as 10 implementações.

1. Introdução

A proliferação de LLMs de grande escala — modelos com dezenas a centenas de bilhões de parâmetros — trouxe à tona um gargalo clássico: a largura de banda de memória. Durante a fase de inferência, matrizes de peso extremamente esparsas (resultado de técnicas de pruning estruturado e não-estruturado com esparsidades de 30% a 90%) precisam ser multiplicadas por vetores de ativação de forma repetida e eficiente. Nesse contexto, a operação SpMV (Sparse Matrix-Vector Multiplication) torna-se um dos kernels dominantes, e a escolha do formato de armazenamento esparsos tem impacto direto sobre a taxa de transferência efetiva e o tempo de inferência.

Formatos clássicos como CSR (Compressed Sparse Row) apresentam padrões de acesso à memória não coalescidos em GPU — um problema estrutural que reduz drasticamente a utilização da largura de banda disponível. Formatos especializados como o MACKO (2024/2025) foram propostos para endereçar esse problema ao alinhar dados de coluna e valor em blocos de memória contíguos (chunks), reduzindo indireções e melhorando a coalescência de acesso.

Este trabalho não replica o MACKO: propõe uma implementação independente inspirada nos seus princípios, com contribuições originais em três dimensões: (i) delta encoding com compressão de metadados de coluna de int32 para int16 por meio de endereçamento relativo dentro de cada chunk; (ii) refatoração do layout de AoS para SoA para eliminar o padding silencioso de struct imposto pelo alinhamento de double e maximizar a coalescência de warp; e (iii) uma variante de precisão mista que armazena os valores em FP16 no device e realiza a acumulação em FP64, reduzindo o footprint de vals de 32 para 8 bytes por chunk.

O objetivo é superar o baseline CSR-CUDA em throughput de GB/s em toda a faixa de esparsidades relevante para LLMs podados, com um harness de benchmark rigoroso que inclui CSR sequencial, CSR OpenMP, SP24, CSR-CUDA e cuSPARSE como controles.

2. Formulação do Problema

Dado um vetor denso $x \in \mathbb{R}^n$ e uma matriz esparsa $A \in \mathbb{R}^{n \times n}$ com nnz elementos não-nulos, deseja-se computar: $y = A \cdot x$. Essa operação é memory-bound: a razão de aritmética ($2 \cdot \text{nnz}$ operações de ponto flutuante) sobre bytes transferidos é tipicamente inferior a 1 FLOP/byte, limitando o desempenho pela largura de banda do subsistema de memória.

O problema específico aqui considerado é a SpMV em matrizes quadradas de esparsidade variável (30%–90%), geradas sinteticamente para representar matrizes de peso de LLMs podados. As dimensões são $n \in \{4096, 16384, 32768\}$, cobrindo desde camadas de atenção pequenas até matrizes de projeção de modelos de grande porte.

A métrica primária de avaliação é GB/s efetivos, calculados como: $\text{GB/s} = (\text{bytes_lidos} + \text{bytes_escritos}) / \text{tempo_de_kernel}$. Os bytes são computados com base no footprint real do formato, incluindo metadados de linha, coluna e valores. A métrica secundária é GFLOP/s ($2 \cdot \text{nnz} / \text{tempo}$). Corretude é verificada por comparação com o CSR sequencial com tolerância de 10^{-6} (GPU) e 10^{-9} (CPU).

3. Objetivos

Os objetivos específicos deste trabalho são:

1. Implementar um formato de armazenamento esparsa customizado (MACKO-SoA-FP16) em C+CUDA e OpenMP, partindo dos princípios de chunking do formato MACKO mas com contribuições originais de layout e compressão.
2. Avaliar três versões evolutivas do formato (v1 AoS int32; v2 AoS int16+máscara; v3 SoA), identificando os trade-offs reais de cada escolha de implementação através de medição empírica.
3. Implementar uma variante de precisão mista FP16/FP64 e medir seu impacto em throughput e corretude.
4. Superar o baseline CSR-CUDA ingênuo em GB/s na faixa completa de esparsidades testadas, e aproximar-se ou superar o cuSPARSE nas configurações de alta esparsidade.
5. Documentar a evolução do formato de forma rigorosa, com grupo de controle (CSR e SP24, nunca modificados) para separar ruído de variação real.

4. Metodologia

4.1 Ambiente de Execução

Todos os experimentos foram realizados no laboratório de informática da UFG, em uma máquina equipada com GPU NVIDIA RTX 4090 (arquitetura Ada Lovelace, sm_89, 24 GB GDDR6X) e CUDA Toolkit 12.0. A compilação utilizou `nvcc -arch=sm_89 -O3` para CUDA e `gcc -O3 -fopenmp` para CPU. A medição de tempo GPU foi realizada com `cudaEvent`, confirmada via Nsight Systems com diferença <2% em relação ao CUPTI. A medição de tempo CPU utilizou `clock_gettime(CLOCK_MONOTONIC)`.

4.2 Gerador de Matrizes

Matrizes esparsas quadradas foram geradas sinteticamente com padrão de aleatoriedade uniforme, controlando a fração de zeros pela esparsidade declarada. Cada configuração foi rodada 101 vezes, com o primeiro resultado (warm-up) descartado, resultando em 100 amostras para cômputo de médias, mínimo e máximo.

4.3 Formatos Implementados

4.3.1 CSR (Baseline)

O formato Compressed Sparse Row padrão serve como baseline obrigatório. Para cada linha i , armazena os índices de coluna e valores dos nnz_i elementos não-nulos em arrays contíguos, com um `row_ptr[i]` apontando para o início de cada linha.

4.3.2 SP24

Formato com pruning estruturado 2:4: preserva exatamente os 2 elementos de maior valor absoluto em cada grupo de 4. Permite compressão a 50% do tamanho e uso de instruções SIMD especializadas em CPU. Serve como baseline adicional por representar uma estratégia de compressão estruturada.

4.3.3 cuSPARSE

Biblioteca oficial da NVIDIA para SpMV em GPU, usando a função `cusparsSpMV` com formato CSR. Serve como teto de referência industrial.

4.3.4 MACKO v1 — AoS com int32 (Baseline do Formato Customizado)

Cada linha é dividida em chunks de `CHUNK_SIZE=4` elementos. Um chunk armazena pares de coluna absoluta (`int32`) e valor (`double`), com `sizeof(MACKOChunk) = 48` bytes. O kernel GPU utiliza estratégia warp-per-row: cada warp (32 lanes) processa uma linha da matriz, com cada lane iterando sobre chunks não sobrepostos e acumulação via `__shfl_down_sync`.

```
typedef struct {
    int    cols[4];    // colunas absolutas, int32
    double vals[4];    // valores, FP64
} MACKOChunk; // sizeof = 48 bytes
```

4.3.5 MACKO v2 — AoS com int16 e valid_mask

Motivação: delta encoding relativo a uma coluna base pode reduzir os 16 bytes de índices `int32` para 10 bytes. A v2 introduz `col_base` (`int16`), `col_delta[4]` (`int16`) e `valid_mask` (`uint8_t`).

Resultado observado: regressão de desempenho em todas as três implementações. A causa é o alinhamento de struct em C: `double vals[4]` exige alinhamento de 8 bytes para o struct inteiro, e o compilador insere silenciosamente 5 bytes de padding, resultando em `sizeof(MACKOChunk) = 48` bytes — idêntico à v1. O ganho de compressão é completamente anulado, enquanto o custo computacional de decodificação (`col_base + col_delta[k]`) é somado. Essa regressão demonstra empiricamente que otimizações de compressão de metadados só compensam quando o footprint é efetivamente reduzido na memória.

4.3.6 MACKO v3 — SoA (Contribuição Principal)

A solução para o problema de padding da v2 é a refatoração do layout de AoS (Array of Structs) para SoA (Structure of Arrays):

```
typedef struct {
```

```

int      rows, cols, nnz, total_chunks;
uint8_t *valid_masks; // [total_chunks] - 1 byte/chunk
int16_t *col_bases;   // [total_chunks] - 2 bytes/chunk
int16_t *col_deltas;  // [total_chunks x CHUNK_SIZE] - 8 bytes/chunk
double  *vals;        // [total_chunks x CHUNK_SIZE] - 32 bytes/chunk
int      *row_ptr;
int      *row_nchunks;
} MACKOMatrix;

```

Com SoA, cada array tem seu próprio alinhamento natural, sem interferência entre campos. O footprint por chunk é: $1 + 2 + 8 + 32 = 43$ bytes/chunk, contra 48 da v1 — redução de ~10,4%. Na GPU, as 32 lanes de um warp leem `valid_masks[chunk_idx]` de endereços consecutivos na mesma instrução, gerando uma única transação de memória coalescida de 32 bytes. No layout AoS, o stride entre leituras de lanes vizinhas era de 48 bytes, exigindo múltiplas transações.

4.3.7 MACKO v3 FP16 — Precisão Mista

Variante adicional onde os valores são armazenados em FP16 (`__half`, 2 bytes) no device em vez de FP64 (`double`, 8 bytes). O footprint de `vals` é reduzido de 32 para 8 bytes por chunk, e o footprint total de dados cai de 43 para 19 bytes/chunk. A multiplicação é realizada em FP64 após conversão on-the-fly (`__half2double`), e a acumulação permanece em `double`, preservando a qualidade numérica para verificação com tolerância de 10^{-6} .

4.4 Kernels CUDA

Dois kernels foram implementados para o formato MACKO. O Kernel 1 (thread per row) destina uma thread por linha, iterando sequencialmente sobre todos os seus chunks. O Kernel 2 (warp per row) — adotado como padrão — destina um warp de 32 lanes por linha, com distribuição de chunks entre lanes e redução via `__shfl_down_sync`. Dimensionamento: $\text{tpb}=256$ (8 warps/bloco), $\text{blocks} = (\text{rows} \times 32 + \text{tpb} - 1) / \text{tpb}$.

4.5 Harness de Benchmark

O harness implementado em `benchmark.cu` mede 10 implementações em cada configuração: (1) CPU sequencial CSR, (2) CPU OpenMP CSR, (3) CPU sequencial SP24, (4) CPU OpenMP SP24, (5) GPU CUDA kernel CSR, (6) GPU CUDA cuSPARSE, (7) CPU sequencial MACKO, (8) CPU OpenMP MACKO, (9) GPU CUDA warp/row MACKO v3 SoA FP64, e (10) GPU warp/row MACKO FP16. CSR e SP24 (itens 1–6) nunca foram modificados durante o desenvolvimento, servindo como grupo de controle para distinguir ruído de regressão real entre versões.

5. Resultados

5.1 Estabilidade do Ambiente e Validação da Medição

A variação dos implementações de controle (CSR sequencial e GPU CSR) entre todas as rodadas foi inferior a 2%, confirmando a estabilidade do ambiente de medição. O Nsight Systems (nsys) validou independentemente os tempos de kernel: o tempo de `kernel_macko_warp_per_row` medido via `cudaEvent` (17 μ s) concordou com o tempo reportado pelo CUPTI (16,07 μ s), diferença de <6%. Tentativas de profiling com `ncu` (Nsight Compute) falharam por restrição de permissão de hardware counters nas máquinas disponibilizadas para teste (4090 do laboratório 150 do INF).

5.2 Evolução das Três Versões (n=4096, s=0,85)

Implementação	v1 AoS int32	v2 AoS int16+mask	v3 SoA	v3 vs. v1
Bytes/chunk (real)	48	48 (padding anula)	43	-10,4%
CPU sequencial	1,305 ms	1,426 ms (+9%)	1,226 ms	-6% ✓
CPU OpenMP	0,107 ms	0,128 ms (+20%)	0,134 ms	+25% ✗
GPU warp/row	0,017 ms / 1786 GB/s	0,020 ms / 1549 GB/s	0,014 ms / 1886 GB/s	-18% ✓

Tabela 1 — Evolução das três versões do formato MACKO (n=4096, esparsidade=0,85, 100 amostras)

A v2 regrediu em todos os casos devido ao padding silencioso do compilador, confirmando empiricamente que delta encoding em AoS é neutro em footprint quando há campos de alinhamento grande (double) no mesmo struct. A v3 SoA recuperou o ganho teórico no caso da GPU (coalescência de warp e redução de footprint se somam) e no CPU sequencial. O CPU OpenMP não se beneficiou: em SoA, cada thread abre 4 streams de memória simultâneos, aumentando a pressão no subsistema de memória quando múltiplas threads competem em paralelo.

5.3 Resultados Completos — n=4096

Esparsidade 0,30 (nnz ≈ 11,7M)

Implementação	Méd (ms)	GFLOP/s	GB/s
CPU sequencial (CSR)	6,856	3,43	20,57
CPU OpenMP (CSR)	2,439	9,63	57,81
GPU CUDA kernel (CSR)	0,640	36,71	220,39
GPU CUDA cuSPARSE (CSR)	0,156	150,23	901,91
GPU CUDA warp/row (MACKO)	0,138	169,83	914,05
GPU warp/row (MACKO FP16)	0,045	517,32	1231,44

Tabela 2 — n=4096, esparsidade=0,30

Esparsidade 0,70 (nnz ≈ 5,0M)

Implementação	Méd (ms)	GFLOP/s	GB/s
GPU CUDA kernel (CSR)	0,131	76,75	461,10
GPU CUDA cuSPARSE (CSR)	0,044	230,51	1384,94
GPU CUDA warp/row (MACKO)	0,021	477,90	2576,56
GPU warp/row (MACKO FP16)	0,024	416,07	993,45

Tabela 3 — n=4096, esparsidade=0,70 (apenas implementações GPU)

Esparsidade 0,85 (nnz ≈ 2,5M)

Implementação	Méd (ms)	GFLOP/s	GB/s
GPU CUDA kernel (CSR)	0,070	71,79	431,92
GPU CUDA cuSPARSE (CSR)	0,030	165,90	998,11
GPU CUDA warp/row (MACKO)	0,014	350,35	1894,59
GPU warp/row (MACKO FP16)	0,015	327,37	785,80

Tabela 4 — $n=4096$, esparsidade=0,85 (apenas implementações GPU)

5.4 Resultados em Larga Escala — $n=16384$ e $n=32768$

Implementação	Méd (ms)	GFLOP/s	GB/s
GPU CUDA kernel (CSR)	3,660	44,01	264,15
GPU CUDA cuSPARSE (CSR)	1,017	158,42	950,84
GPU CUDA warp/row (MACKO)	0,928	173,49	933,23
GPU warp/row (MACKO FP16)	0,475	338,77	805,64

Tabela 5 — $n=16384$, esparsidade=0,70 (apenas implementações GPU)

Implementação	Méd (ms)	GFLOP/s	GB/s
GPU CUDA cuSPARSE (CSR)	6,730	159,53	957,30
GPU CUDA warp/row (MACKO)	6,109	175,76	944,94
GPU warp/row (MACKO FP16)	3,216	333,88	793,27

Tabela 6 — $n=32768$, esparsidade=0,50 (apenas implementações GPU)

5.5 Speedup Geral

Tomando o GPU CUDA kernel (CSR) como baseline de comparação primário:

Configuração	Speedup MACKO FP64 vs CSR-CUDA	Speedup MACKO FP16 vs CSR-CUDA
$n=4096$, $s=0,30$	4,6×	14,2×
$n=4096$, $s=0,70$	6,2×	5,5×
$n=4096$, $s=0,85$	5,0×	4,7×
$n=4096$, $s=0,90$	3,5×	3,1×
$n=16384$, $s=0,70$	3,9×	7,7×
$n=32768$, $s=0,50$	3,9×	7,4×

Tabela 7 — Speedup do formato MACKO vs. CSR-CUDA ingênuo

5.6 Comparação com cuSPARSE

Em várias configurações de alta esparsidade com $n=4096$, o formato MACKO v3 SoA apresentou tempo de kernel absoluto menor que o cuSPARSE (ex: $s=0,85$: MACKO 0,014 ms vs. cuSPARSE 0,030 ms — 2,1× mais rápido). Em configurações de baixa esparsidade ($s=0,30$) e matrizes grandes ($n=16384$, $n=32768$), a versão FP16 supera o cuSPARSE tanto em tempo absoluto quanto em GFLOP/s efetivos.

6. Conclusão

Este trabalho desenvolveu e avaliou um formato de armazenamento esparsa customizado para SpMV aplicado a LLMs podados, com três contribuições originais além da mera replicação do formato MACKO: (i) um processo iterativo de design documentado com grupo de controle, que revelou empiricamente a armadilha do padding silencioso em AoS com campos de alinhamento heterogêneo; (ii) a refatoração para SoA como solução que recupera simultaneamente o ganho de footprint e a coalescência de warp em GPU; e (iii) uma variante FP16 que reduz o footprint de valores em 4× e entrega speedups de até 14× sobre o CSR-CUDA ingênuo em configurações de baixa esparsidade.

Os resultados confirmam que a estratégia warp-per-row com layout SoA é altamente eficaz em GPUs modernas: a coalescência de warp torna o acesso a campos leves (`uint8_t`, `int16_t`) de chunks consecutivos praticamente sem custo de largura de banda de metadados, concentrando quase toda a pressão de memória nos vals — que o FP16 reduz a 25% do tamanho original.

As lições principais do projeto são: (1) medir antes de assumir — a v2 parecia uma otimização óbvia no papel e se revelou uma regressão real ao executar; (2) SoA não é uma otimização universal — ela depende do padrão de acesso predominante (mesmo campo de elementos vizinhos para GPU vs. todos os campos de um elemento para CPU OpenMP); e (3) um grupo de controle intocado é indispensável para distinguir variação real de ruído de medição.

Como trabalhos futuros, aponta-se: (a) avaliação com matrizes de peso reais extraídas de modelos podados (LLaMA-2, Mistral); (b) implementação de uma variante com metadados de coluna em INT8; (c) exploração de tensor cores com formato de dados compatível para acumulação em FP16; e (d) avaliação em múltiplas GPUs com NVLink para cargas de inferência em batch.

Referências

1. NVIDIA Corporation. cuSPARSE Library User Guide. CUDA Toolkit Documentation, 2024.
2. NVIDIA Corporation. CUDA C++ Programming Guide. 2024.
3. Han, S. et al. Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding. ICLR, 2016.
4. Mishra, A. et al. Accelerating Sparse Deep Neural Networks. arXiv:2104.08378, 2021.
5. Frantar, E.; Alistarh, D. SparseGPT: Massive Language Models Can Be Accurately Pruned in One Shot. ICML, 2023.
6. Gale, T. et al. Sparse GPU Kernels for Deep Learning. SC'20, 2020.
7. Macko, Vladimír, and Vladimír Boža. "MACKO: Sparse Matrix-Vector Multiplication for Low Sparsity." *arXiv preprint arXiv:2511.13061* (2025).
8. Bell, N.; Garland, M. Implementing Sparse Matrix-Vector Multiplication on Throughput-Oriented Processors. SC'09, 2009.